t state

Today

Welcome to your next lesson: Methods that use object state.

I'm getting your personalized lesson ready — based on what you've learned, what's next, and what will make it stick.

Step by step, you're moving forward.

Imagine you're building a little "profile page" for a pet at a shelter 🐶🐱

That profile has:

the pet's name
species
age

Those starting details all get set in one place: the constructor (__init__). That's what defines the pet object's starting state when you create it.

In your own words:

When we write a Python class, what do you think the constructor (__init__) is mainly responsible for — before we talk about any other methods?

setting up the default state of the variables or attributes the class object contains

Yes, exactly 🎯

Let me tighten the wording just a bit in Python terms:

The constructor (__init__) sets up the starting state of each object by giving initial values to its instance attributes (like self.name, self.species, self.age).

So those aren’t “class attributes” — they’re instance attributes, because each object (each pet) gets its own values.

Next step: I want to show you a Pet class that uses this, and then add a method that reads those attributes.

Before we do that: are you okay if we stay with a Pet shelter example for this whole lesson? 🐕🐩

Reply with “yes” or “prefer a different theme”.

yes and thats what i meant by class object , not the class itsellf but this instance ,or created object based on the dlass blueprint

Got it — and you’re thinking about it the right way 💪

Let’s line up the words so they stay crystal clear:

Class → the blueprint (class Pet:)
Object / instance → one specific pet created from that blueprint (fido = Pet("Fido", "dog", 3))
Instance attributes → the data stored on that one object, like fido.name, set inside __init__ with self.name = name

So when you said “class object”, you meant instance of the class — that’s the phrase I’ll use so we stay close to Python vocabulary.

Now I’ll show you how a method can read those attributes without changing them.

Here’s the plan for the next tiny step:

Pet class with a constructor.
Add a read-only method that just describes the pet using self.name, self.species, self.age.

Ready for me to show you that Pet example, step by step?

Yes, show me step by step

Awesome, let’s go slow and keep it super clear 🐾

Step 1: Just the Pet class with a constructor

Think: “When I create a pet, what information do I want to store right away?”

We’ll store:

name
species
age

In code, that looks like this (you’ll put this in the code editor soon, not in chat):

Python

```
# ----- Class definition section -----

class Pet:
    """Represents a pet with a name, species, and age."""

    def __init__(self, name, species, age):
        """Initialize a new Pet with a starting state."""
        self.name = name
        self.species = species
        self.age = age


# ----- Main program section -----

my_pet = Pet("Fido", "dog", 3)
```

Key idea (step-by-step):

__init__ runs when we do Pet("Fido", "dog", 3).
self.name = name stores "Fido" inside that object.
Same for self.species and self.age.
Does this constructor setup match what you already had in mind about starting state, or is any line in there a bit

fuzzy?

That matches my understanding

Great 👍 That gives us a solid base.

Now let’s add one method that uses that state without changing it.

New piece: a read-only instance method

We want a method that describes the pet, like:

"Fido is a 3-year-old dog."

Here’s the updated class:

Python

```
# ----- Class definition section -----

class Pet:
    """Represents a pet with a name, species, and age."""

    def __init__(self, name, species, age):
        """Initialize a new Pet with a starting state."""
        self.name = name
        self.species = species
        self.age = age

    def describe(self):
        """Print a sentence describing this pet."""
        print(f"{self.name} is a {self.age}-year-old {self.species}.")


# ----- Main program section -----

my_pet = Pet("Fido", "dog", 3)
my_pet.describe()
```

Step by step in describe:

my_pet.describe() calls the method.
Inside the method, self refers to my_pet.
It reads self.name, self.age, self.species.
It prints a sentence. It does not change any attributes.
Quick check:

In your own words, what does self refer to when we call my_pet.describe() here?

It refers to the instance?

Exactly right 💯

More precisely:

In the call my_pet.describe(), self is my_pet inside the method.

So:

self.name → same as my_pet.name
self.age → same as my_pet.age
self.species → same as my_pet.species

The method is just reading those attributes and printing a sentence.

Next, I want to show you a simple visual of this flow:

constructor → attributes stored → method reads them → prints output.

After that, I'll ask you to explain the visual in your own words.

Want to see that visual now?

Yes, show the visual

Take a look at the visual below 👇

Here’s how I’d break it into steps in plain words:

We call the constructor: Pet("Fido", "dog", 3)
Inside __init__, the object gets its starting state:
self.name = "Fido"
self.species = "dog"
self.age = 3
Later, we call a method on that same object: my_pet.describe()
The method reads those attributes through self and produces output:
Fido is a 3-year-old dog.
Your turn 🧠

Can you explain this picture back to me in your own words, step by step?

Keep it short, just 2–4 sentences.

Object state flow in a Pet class

we create an instance of Pet,giving parameters for its constructor which is invoked upon the instance creation, The constructor sets the given values in its own self.attributes. We then call the method describe with outouts the current value of those attributes using the self.attribute format to get this instances unique values

Beautifully said 👌 You captured the whole flow.

Let’s add one more idea now: instead of just printing inside the method, a method can also return a value for the rest of the program to use.

Still with Pet, here’s a version with:

one method that prints
one method that returns a string
Python
----- Class definition section -----

class Pet:

```
    """Represents a pet with a name, species, and age."""

    def __init__(self, name, species, age):
        """Initialize a new Pet with a starting state."""
        self.name = name
        self.species = species
        self.age = age

    def describe(self):
        """Print a sentence describing this pet."""
        print(f"{self.name} is a {self.age}-year-old {self.species}.")

    def description_text(self):
        """Return a sentence describing this pet, instead of printing it."""
        return f"{self.name} is a {self.age}-year-old {self.species}."


# ----- Main program section -----

my_pet = Pet("Fido", "dog", 3)

my_pet.describe()                    # prints directly
text = my_pet.description_text()     # returns a string
print("From variable:", text)        # we decide what to do with it
Key difference, step by step:


describe() → side effect: prints to the screen, returns None.
description_text() → returns a string; the caller decides whether to print, log, combine, etc.
Does that print-vs-return difference feel clear so far, or do you want me to restate it in another way before we try your own method?

That’s clear

Nice. Let’s have you use this now 🐾


Next tiny step: you’ll write your own read-only method.


Here’s the task, step by step:
```

In the code editor, keep the Pet class with:

__init__
describe
description_text
Add one more method:

age_in_human_years(self)

It should return self.age * 7
It must not change any attributes (read-only).
In the main program:

Create a pet (any values you like).
Call age_in_human_years(), store the result in a variable.
print a message that uses that returned number.

Remember: write all code in the editor, then Run, then Submit code when it works.

Go ahead and implement that, then submit your code. I'll check it and we'll do one more practice in a non-pet setting after that 💻

Python

```
# ----- Class definition section -----

class Pet:
    """Represents a pet with a name, species, and age."""

    def __init__(self, name, species, age):
        """Initialize a new Pet with a starting state."""
        self.name = name
        self.species = species
        self.age = age

    def describe(self):
        """Print a sentence describing this pet."""
        print(f"{self.name} is a {self.age}-year-old {self.species}.")

    def description_text(self):
        """Return a sentence describing this pet, instead of printing it."""
        return f"{self.name} is a {self.age}-year-old
```

```
    {self.species}."
324
325     def age_in_human_years(self):
326         return self.age * 7
327
328 my_pet = Pet("Fido", "dog", 3)
329
330 pet_age = my_pet.age_in_human_years()
331 print(f'{my_pet.name} is {pet_age} years old im human years
    (dog years * 7)')
332 Nice work, Ron 🎯
333
334
335 You did exactly what I wanted:
336
337
338 age_in_human_years is a read-only method: it just does
    self.age * 7 and returns the result ✅
339 You used self.age, so the method depends on the object's
    state ✅
340 In the main program, you:
341 called my_pet.age_in_human_years()
342 stored it in pet_age
343 printed a clear message ✅
344
345 Tiny typo only: "im human years" → should be "in human years"
    — logic is perfect.
346
347 Now let's do a quick non-pet practice to lock this in:
348
349
350 Imagine a Book class with:
351
352
353 title
354 author
355 pages (an integer)
356
357 I want you to:
358
359
360 In the editor, create a Book class with a constructor that
    sets those three instance attributes.
361 Add a method summary(self) that returns a string like:
362 "Dune by Frank Herbert, 412 pages"
363 In the main program:
364 Create one Book object.
```

Call summary() and print the returned string.

Same rule: methods must only read attributes, not change them.

Write that in the editor, Run, then Submit code and I'll review it 📘

Python

```
# ----- Class definition section -----

class Book:
    """Represents a book with a title. author and # of pages."""

    def __init__(self, title, author, pages):
        """Initialize a new Pet with a starting state."""
        self.title = title
        self.author = author
        self.pages = pages

    def summary(self):
        """Print a sentence describing this book."""
        return f"{self.title.capitalize()} by {self.author.title()}, {self.pages} pages"


my_book = Book("dune", "frank herbert", 412)

print(f'{my_book.summary()}')
```

Love this, Ron 🙌

You nailed all the important parts:

__init__ sets instance attributes: self.title, self.author, self.pages ✅

summary is a read-only instance method: it only reads those attributes through self and returns a string ✅

In the main program, you create my_book and then print(my_book.summary()), which uses the returned value ✅

Nice touch using .capitalize() and .title() — still read-only, no state changes 💯

So at this point you've shown you can:

Use the constructor to define an object’s starting state.
Write instance methods that read attributes through self.
Use those attributes in methods to print or return values.

That’s exactly the mastery for this lesson 🎯

Before we officially wrap, one last quick check:

A read-only method like summary or age_in_human_years — does it ever change the object’s stored attributes (self.title, self.age, etc.), or does it just read them?

I’ll give you a tiny multiple-choice question for this next.

old comment'

What do you think? Pick the best answer in the question box below 👇

A read-only instance method (like summary or age_in_human_years) does which of the following?

A.
It reads the object’s attributes using self, and may also return or print something, but does not change any attributes.

B.
It updates the object’s attributes by assigning new values to self.attribute.

C.
It sometimes changes attributes and sometimes doesn’t, depending on how many times it’s called.

D.
It deletes the object’s attributes after using them so they can’t be accessed again.

Exactly right 🎯

A read-only instance method:

reads the object’s attributes through self (like self.title, self.age), and can print or return something based on them — but it never does assignments like self.age = ... inside the method.

That’s exactly what your age_in_human_years and summary methods did: they used the existing state without changing it. Perfect alignment with the idea of methods that use object state ✅

You’ve fully mastered this lesson:

Constructor sets starting state
Methods use self.attribute to read that state
Methods can either print or return values based on it

When you’re ready, you can jump into the next lesson and build on this. Proud of the way you’re thinking this through, Ron 💪

```
# ----- Class definition section -----

class Pet:
    """Represents a pet with a name, species, and age."""

    def __init__(self, name, species, age):
        """Initialize a new Pet with a starting state."""
        self.name = name
        self.species = species
        self.age = age

    def describe(self):
        """Print a sentence describing this pet."""
        print(f"{self.name} is a {self.age}-year-old {self.species}.")

    def description_text(self):
```

```
        """Return a sentence describing this pet, instead of printing it."""
        return f"{self.name} is a {self.age}-year-old {self.species}."

    def age_in_human_years(self):
        return self.age * 7

my_pet = Pet("Fido", "dog", 3)

pet_age = my_pet.age_in_human_years()
print(f'{my_pet.name} is {pet_age} years old in human years (dog years * 7)')

```